

```
In [ ]: class PIVelocityController:
    """
    PI velocity controller.
    Maps velocity error to a PWM percentage (pwm command)
    """

    def __init__(self, K_p: float, K_i: float, max_pwm_percent: float = 100.0):
        self.K_p = float(K_p)
        self.K_i = float(K_i)
        self.max_pwm = float(max_pwm_percent)

        self.error_prev = 0.0
        self.error_integral = 0.0

    def update(self, target_vel_rad_s: float, actual_vel_rad_s: float, dt: float):
        error = target_vel_rad_s - actual_vel_rad_s

        # Tustin Discretization for the Integral Term
        #  $I[k] = I[k-1] + (K_i * dt / 2) * (e[k] + e[k-1])$ 
        self.error_integral += (self.K_i * dt / 2.0) * (error + self.error_prev)

        # Anti-windup: clamp the integral term
        self.error_integral = max(min(self.error_integral, self.max_pwm), -self.max_pwm)

        # Calculate total PWM and clamp final output
        pwm_cmd = (self.K_p * error) + self.error_integral
        pwm_cmd = max(min(pwm_cmd, self.max_pwm), -self.max_pwm)

        # Save state for the next discrete step
        self.error_prev = error

        return pwm_cmd
```

```
In [ ]: class AdmittanceFishingController:
    def __init__(self, M_v, B_v, pi_Kp, pi_Ki, desired_tension_N, L, dxl_id, control_freq_Hz, max_duration_s):
        self.M_v = float(M_v) # Virtual Mass
        self.B_v = float(B_v) # Virtual Damper
        self.desired_tension_N = float(desired_tension_N)
        self.L = float(L) # Length of Rod
        self.dxl_id = dxl_id # Dynamixel ID

        self.dt = 1.0 / float(control_freq_Hz)
        self.max_duration_s = float(max_duration_s)
        self.loop_manager = FixedFrequencyLoopManager(control_freq_Hz)

        # Tustin discretization
        # Continuous domain:  $V(s)/E(s) = 1 / (M_v*s + B_v)$ 
        # Applying Tustin transform:  $s = (2/dt) * (1 - z^{-1}) / (1 + z^{-1})$ 
        self._tustin_a = (2.0 * self.M_v / self.dt) + self.B_v
        self._tustin_b = (2.0 * self.M_v / self.dt) - self.B_v

        # State history for the discrete filter
```

```

self._e_prev = 0.0
self._v_prev = 0.0

# Data Logging
self.tension_history = []
self.velocity_cmd_history = []
self.time_stamps = []

# Instantiate the inner loop PI controller
self.pi_controller = PIVelocityController(pi_Kp, pi_Ki, max_pwm_perc

self.prev_time = time.time()

# - Sensor Reads -
def _read_load_cell(self):
    """
    Reads line tension
    """
    # TODO: READ SENSOR DO THIS IDK HOW
    tension_measured
    return tension_measured

def _read_motor_state(self):
    """
    Reads present position and velocity from the Dynamixel.
    """
    # I took this from minilab 2 mostly idk if it's how you do this
    groupSyncReadState.txRxPacket()

    v_ticks = groupSyncReadState.getData(self.dxl_id, ADDR_PRESENT_VELOC
    p_ticks = groupSyncReadState.getData(self.dxl_id, ADDR_PRESENT_POSIT

    q_rad = pos_ticks_to_rad(p_ticks)
    qdot_rad_per_s = vel_ticks_to_rad_per_s(v_ticks)

    return q_rad, qdot_rad_per_s

def _apply_pwm_percent(self, pct: float):
    # IDK if this is right or if we even need this im ngl
    """Write the scaled goal-PWM register to the motor."""
    pct = max(min(pct, 100.0), -100.0)
    pwm_raw = int(round((pct / 100.0) * PWM_FULL_SCALE))
    if pwm_raw < 0:
        pwm_raw += 1 << 16
    packetHandler.write2ByteTxRx(portHandler, self.dxl_id, ADDR_GOAL_PWM

def control_loop(self):
    print(f"Starting Cascaded Admittance Loop. Target: {self.desired_ter
    t_start = time.time()
    self.prev_time = t_start

    while True:
        now = time.time()
        dt = now - self.prev_time if now > self.prev_time else self.dt
        self.prev_time = now

```

```

t = now - t_start

if t >= self.max_duration_s:
    break

# 1. Gather Sensor Feedback
q_rad, actual_vel_rad_s = self._read_motor_state()
tension_actual_N = self._read_load_cell()

# 2. Outer Loop: Admittance (Force -> Target Velocity)
force_error = self.desired_tension_N - tension_actual_N
torque_error = force_error * self.L

v_cmd = (1.0 / self._tustin_a) * (torque_error + self._e_prev +

self._e_prev = torque_error
self._v_prev = v_cmd

# 3. Inner Loop: PI Control (Velocity Error -> PWM Command)
pwm_cmd = self.pi_controller.update(
    target_vel_rad_s=v_cmd,
    actual_vel_rad_s=actual_vel_rad_s,
    dt=dt
)

# 4. Command Actuator
self._apply_pwm_percent(pwm_cmd)

self.loop_manager.sleep()

```